

CAPÍTULO 2

ESTRUCTURA DE DATOS

LISTAS ENLAZADAS

Estructuras lineales y referencias

Contactos y tickets paso a paso en Python

EDICIÓN PARA ESTUDIANTES

Conceptos, ejemplos y práctica guiada



NODOS

Datos y referencias



ENLACES

Conectar y recorrer



PYTHON

Construir y practicar



Estructura de Datos

Capítulo 2 · Listas enlazadas

Módulo 2 – Linked Lists: Linear Data Structures

Manual de estudio y práctica con Python

Basado en la presentación «Listas Enlazadas: Fundamentos, Tipos y Complejidad», de Ing. Leonardo Tavaréz, y en las dos clases prácticas suministradas: agenda de contactos y sistema de tickets.

El contenido se organiza como un capítulo de libro: explica los conceptos, reconstruye las operaciones paso a paso y ofrece implementaciones completas para estudiar con el editor abierto.

OBJETIVO DEL CAPÍTULO

Construir y verificar listas simples y dobles, razonar sobre sus enlaces y distinguir el costo de localizar un nodo del costo de modificar la estructura.

Conocimientos previos. Variables, condicionales, ciclos, funciones y nociones de clases. Se repasan las referencias y las herramientas de Python utilizadas.

Herramientas. Python 3.10 o posterior, editor y terminal. Sin paquetes externos.

Incluye dos proyectos, cinco diagramas, doce actividades y respuestas comentadas.

Edición de estudio · Septiembre de 2026

Contenido

Estudia primero los fundamentos. Construye `contacts.py` en las secciones 2.7-2.10 y `tickets.py` en 2.11-2.14; integra los menús en 2.15. El índice incluye enlaces internos.

2.1 De los datos a los enlaces	4
2.2 Arreglos y listas: elegir según las operaciones	5
2.3 Anatomía de una lista e invariantes	6
2.4 Simple, doble y circular	7
2.5 Complejidad: localizar no es enlazar	8
2.6 Preparación del proyecto y herramientas de Python	10
2.7 Agenda: nodo, recorrido y búsqueda	11
2.8 Agenda: insertar en los extremos y en medio	13
2.9 Agenda: eliminar sin perder la secuencia	15
2.10 Agenda: edición segura y primera ejecución	16
2.11 Tickets: modelo y reglas de atención	17
2.12 Tickets: recorridos y búsqueda	20
2.13 Tickets: insertar y conservar los enlaces dobles	21
2.14 Tickets: retirar y editar	23
2.15 Menús de consola: compartir la interacción	24
2.16 Sesión práctica de principio a fin	28
2.17 Comparar el costo real de ambos proyectos	29
2.18 Extensión conceptual: listas circulares	30
2.19 Pruebas: demostrar que los enlaces se conservan	32
2.20 Errores frecuentes y estrategia de depuración	34
2.21 Actividades y respuestas orientativas	35
2.22 Glosario y fuentes del capítulo	38

Listas enlazadas: estructuras lineales

2

Ruta de aprendizaje

Nodos y referencias → invariantes → complejidad → agenda simple → tickets dobles → menús → listas circulares → pruebas y actividades.

2.1 De los datos a los enlaces

Una agenda conserva contactos; una mesa de ayuda conserva solicitudes. En ambos casos necesitamos agregar, encontrar, modificar y retirar registros. La pregunta de este capítulo es cómo representar su orden sin depender de posiciones consecutivas en un arreglo.

Una **lista enlazada** es una secuencia de nodos conectados mediante referencias. Cada nodo guarda un dato, o un conjunto de campos, y al menos un enlace. El orden lógico lo determinan esos enlaces: el siguiente elemento es el objeto al que apunta la referencia next, aunque los objetos no ocupen lugares vecinos en memoria.

Esta organización permite insertar o retirar nodos cambiando unas pocas referencias. Sin embargo, llegar hasta el lugar de la modificación puede requerir recorrer toda la lista. Distinguir esos dos trabajos será una de las ideas centrales del módulo.

2.1.1 Qué podrás hacer al finalizar

- Dibujar listas simples, dobles y circulares e identificar sus condiciones de integridad.
- Implementar inserción, eliminación, búsqueda, edición y recorrido en Python.
- Explicar el efecto de head, tail y prev sobre el costo de las operaciones.
- Construir una agenda de contactos y un gestor de tickets con menús de consola.
- Probar listas vacías, unitarias y con varios nodos, incluyendo operaciones que deben fallar.

2.1.2 Cómo estudiar el capítulo

Lee una operación, dibuja sus enlaces antes y después, escribe el método y ejecuta un caso pequeño. Comprueba después los extremos y el tamaño. Esa rutina ayuda a detectar errores que una impresión superficial puede ocultar.

Los proyectos conservan las funcionalidades de las clases grabadas. El código se reorganiza en tres archivos para que puedas ejecutarlo completo. Las explicaciones incorporan precisiones sobre complejidad, validación y referencias; las actividades finales te permiten aplicar lo aprendido sin depender de los videos.

Actividad de entrada. Imagina una fila de personas en la que cada una solo conoce a quien sigue. ¿Qué información falta para retirar a la última persona sin recorrer la fila? Conserva tu respuesta hasta estudiar la eliminación en una lista simple.

2.2 Arreglos y listas: elegir según las operaciones

Un arreglo almacena sus posiciones en una región contigua. Si conocemos la posición inicial y el tamaño de cada elemento, podemos localizar directamente una posición por su índice. En una lista enlazada, el enlace de un nodo nos conduce al siguiente; no existe ese cálculo directo del nodo número k .

Insertar en medio de un arreglo suele exigir desplazar elementos para abrir espacio. En una lista enlazada, los nodos existentes permanecen donde están: se cambia quién apunta a quién. La ventaja aparece cuando ya disponemos de las referencias necesarias para modificar la estructura.

Criterio	Arreglo dinámico	Lista enlazada
Acceso por índice	$O(1)$.	$O(n)$ en el peor caso.
Inserción intermedia	$O(n)$ por desplazamiento.	$O(1)$ para enlazar; localizar el lugar puede costar $O(n)$.
Memoria adicional	Reserva de capacidad; organización compacta de posiciones.	Nodo y referencias adicionales por elemento.
Recorrido	Buena localidad de memoria en implementaciones habituales.	Salto entre referencias; más indirección.

Un arreglo fijo tiene capacidad determinada. Un arreglo dinámico puede crecer, aunque algunas ampliaciones exigen reservar espacio y copiar referencias o valores. Por eso conviene distinguir el costo de una operación aislada del costo amortizado de muchas operaciones.

En CPython, la lista incorporada `list` es un arreglo dinámico de referencias a objetos. No es la lista enlazada que construiremos aquí. Sus objetos tampoco tienen que estar contiguos, aunque sí lo esté el arreglo interno de referencias. Usaremos `list` en las

pruebas como herramienta de comparación, nunca para almacenar internamente los nodos de nuestros proyectos.

Precisión. Que los datos cambien con frecuencia no basta para preferir una lista enlazada. Importa dónde cambian, cómo se localiza cada elemento y cuánto pesan las búsquedas, el recorrido y la memoria adicional.

2.3 Anatomía de una lista e invariantes

2.3.1 Datos, enlaces y extremos

El **nodo** combina información y conexiones. En la agenda tendrá nombre, correo y teléfono. En el sistema de tickets tendrá identificador, cliente, asunto, descripción y estado. Estos campos describen el dominio; *next* y *prev* describen la estructura que los conecta.

La referencia **head** identifica el primer nodo. La referencia **tail**, cuando la mantenemos, identifica el último. El contador **length** registra cuántos nodos pertenecen a la lista. No es un índice ni una referencia al último nodo.

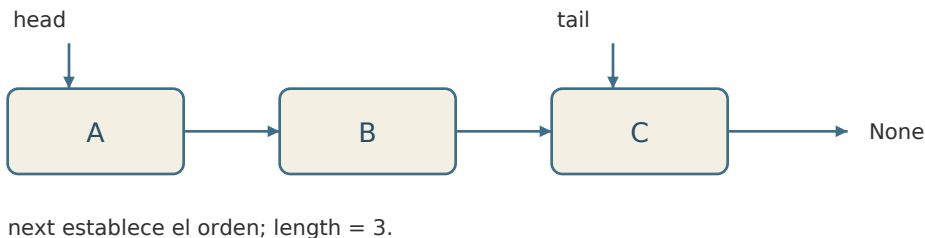


Figura 2.1 · Una lista simple no circular con tres nodos.

Una lista vacía tiene *head* y *tail* iguales a *None* y longitud cero. Una lista de un elemento tiene *head* y *tail* apuntando al mismo objeto. En una lista simple no circular, el *next* del último nodo vale *None*; ese valor marca el final del recorrido.

2.3.2 Invariantes: reglas que siempre deben cumplirse

Una invariante es una condición que debe ser verdadera al terminar cualquier operación pública. No hace falta repetir todas las comprobaciones dentro de cada método, pero sí diseñarlo y probarlo pensando en ellas.

Situación	Regla que debe cumplirse
Lista vacía	head is None, tail is None y length == 0.
Lista no vacía	Ambos extremos existen y length es positivo.
Último nodo	tail.next is None en una lista no circular.
Recorrido completo	Se visitan exactamente length nodos distintos.
Lista simple	Desde head se llega a tail siguiendo next.
Lista doble	head.prev is None y cada enlace tiene su inverso coherente.

Una variable puede cambiar sin mover ningún objeto. Si asignamos `head = head.next`, la cabecera pasa a referirse a otro nodo; no copiamos sus campos ni desplazamos el resto de la secuencia.

2.4 Simple, doble y circular

2.4.1 Lista simplemente enlazada

Cada nodo dispone de `next`. El recorrido natural avanza desde `head` hasta `None`. Para llegar al predecesor de un nodo debemos haber conservado esa referencia o buscarla desde el inicio. Es la estructura del proyecto de contactos.

2.4.2 Lista doblemente enlazada

Cada nodo añade `prev` para señalar al anterior. Si `A.next` es `B`, entonces `B.prev` debe ser `A`. Esta reciprocidad permite recorrer la lista en ambos sentidos y retirar un nodo conocido sin buscar a su predecesor.

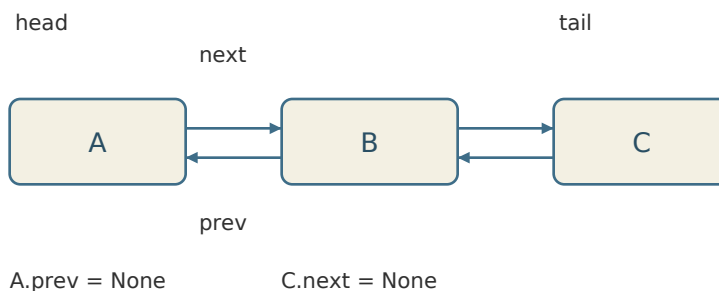


Figura 2.2 · Cada conexión interior tiene su conexión inversa.

La referencia adicional consume memoria y obliga a actualizar más enlaces. Tener prev no vuelve constante la búsqueda por un campo: sin un índice auxiliar, todavía hay que comparar nodos uno por uno.

2.4.3 Lista circular

En una lista circular simple, el siguiente del último nodo es head. En una circular doble, además, el anterior de head es tail. No aparece None como terminación del ciclo no vacío; el recorrido debe detenerse al regresar al inicio o al completar una cantidad conocida de pasos.

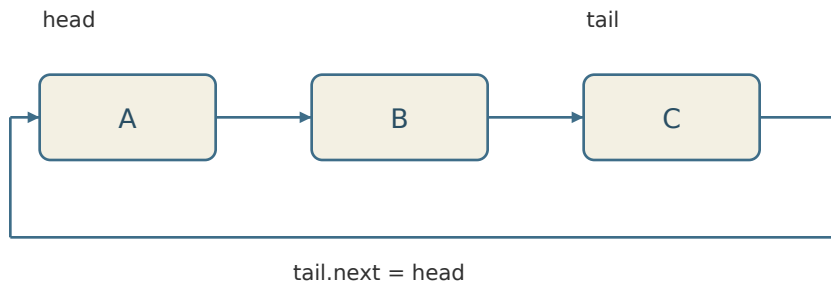


Figura 2.3 · El último enlace regresa al primer nodo.

La circularidad y la cantidad de enlaces son características distintas. Una lista puede ser simple y circular, o doble y circular. Los dos proyectos principales de este manual son **no circulares**. La sección 2.18 explica la adaptación conceptual sin cambiar sus implementaciones.

2.5 Complejidad: localizar no es enlazar

Usaremos n para representar la cantidad de nodos. $O(1)$ describe un número de pasos que no crece con n ; $O(n)$, un trabajo que puede crecer proporcionalmente a n . No se trata de segundos exactos: influyen el equipo, el intérprete y el tamaño de los datos.

Las tablas consideran comparaciones de campos de longitud acotada y el peor caso respecto a n . Procesar o imprimir textos arbitrariamente largos añade un costo proporcional a esos textos. También distinguiremos memoria total $O(n)$ de memoria auxiliar $O(1)$ para un recorrido que conserva unas pocas referencias.

2.5.1 Operaciones estructurales

Operación	Simple con head y tail	Doble con head y tail
Consultar extremos o tamaño almacenado	$O(1)$.	$O(1)$.
Enlazar al principio o al final	$O(1)$.	$O(1)$.
Retirar el primero	$O(1)$.	$O(1)$.
Retirar el último	$O(n)$: falta el anterior.	$O(1)$: tail.prev lo proporciona.
Buscar por valor o acceder por índice	$O(n)$.	$O(n)$.
Insertar después de un nodo conocido	$O(1)$.	$O(1)$.
Eliminar un nodo interior conocido	$O(1)$ si también se conoce su anterior.	$O(1)$ con sus dos enlaces.

En una lista simple que solo conserva head, insertar al final cuesta $O(n)$: hay que encontrar el último nodo. Esta es la situación de la tabla de lista simple en la presentación. Al añadir tail, el enlace final cuesta $O(1)$. La mejora exige mantener tail correctamente después de cada cambio.

2.5.2 El costo de una operación completa

Agregar un contacto exige comprobar que su teléfono no exista. Esa búsqueda cuesta $O(n)$, aunque conectar el nodo al inicio o al final cueste $O(1)$. El método completo tiene costo $O(n)$. Dos recorridos consecutivos siguen siendo $O(n)$, porque $O(n + n)$ se simplifica a $O(n)$.

En el proyecto de tickets también comprobaremos que el UUID generado no coincida con uno existente. La validación lineal vuelve $O(n)$ la creación e inserción completa. Los retiros en los extremos de la lista doble seguirán siendo $O(1)$: no necesitan esa validación.

Precisión. Un menú no tiene complejidad $O(1)$ por mostrar una cantidad fija de opciones. Si el usuario ejecuta m acciones, el costo depende de la suma de esas acciones y del tamaño de la lista en cada una; además, la interacción puede repetirse indefinidamente.

2.6 Preparación del proyecto y herramientas de Python

2.6.1 Carpeta y archivos

Crea una carpeta llamada `modulo2_listas`. Dentro, guarda `contacts.py`, `tickets.py` y `main.py`. Necesitas Python 3.10 o posterior y un editor de texto; el proyecto no requiere instalar paquetes externos. Todos los ejemplos manejan datos en memoria: cerrar el proceso descarta los registros.

Archivo	Contenido
<code>contacts.py</code>	Validador <code>required</code> , <code>ContactNode</code> y <code>ContactLinkedList</code> .
<code>tickets.py</code>	Estados, <code>TicketNode</code> y <code>TicketDoublyLinkedList</code> . Importa <code>required</code> .
<code>main.py</code>	Entrada de datos, menús y función de inicio.
<code>test_lists.py</code>	Comprobaciones reproducibles de la sección 2.19.

Los listados de cada archivo aparecen en orden. Cuando se indique «continúa dentro de la clase», conserva cuatro espacios de sangría antes de cada `def`. No repitas la declaración `class`. Los ejemplos identificados como experimentos o pruebas se ejecutan por separado; no son métodos que debas pegar dentro de las clases.

2.6.2 Anotaciones, `None` e identidad

Python tiene tipado dinámico: los objetos tienen tipos y las variables pueden referirse a objetos de distintos tipos. Las anotaciones ayudan a documentar y a analizar el código; no validan automáticamente los valores durante la ejecución.

`ContactNode | None` expresa que una referencia puede señalar a un nodo o no señalar a ninguno. Es equivalente, en este contexto, a `Optional[ContactNode]`. No significa que el argumento tenga un valor predeterminado: para poder omitirlo en una llamada necesitamos, por ejemplo, `phone=None`.

La instrucción `from __future__ import annotations` permite escribir anotaciones con nombres que aún no se han terminado de definir. Por eso podemos mencionar `ContactNode` dentro de su propia clase. Debe colocarse al comienzo del archivo, después de un posible comentario o docstring de módulo.

Usaremos `is` para comparar identidad: `head is tail` pregunta si ambos nombres señalan al mismo objeto. Usaremos `==` para comparar contenido: `node.phone == phone` pregunta si dos cadenas tienen el mismo valor. Para comprobar `None`, escribe `is None` o `is not None`.

2.6.3 Generadores y atributos permitidos

Un método con `yield` produce un generador: entrega un elemento y conserva el punto donde debe continuar. No construye de antemano otra colección con todos los nodos. Crear el generador no recorre la lista; consumirlo por completo cuesta $O(n)$.

`__slots__` declara los atributos admitidos en nuestras clases de nodos y evita el diccionario de atributos por instancia en estas clases. Puede reducir el consumo de memoria, pero no vuelve inmutables los campos ni comprueba sus tipos. No reemplaza la validación.

2.7 Agenda: nodo, recorrido y búsqueda

2.7.1 Primer bloque de `contacts.py`

El teléfono funcionará como clave única de la agenda. Nombre, correo y teléfono son cadenas no vacías después de retirar espacios exteriores. En este ejercicio no validamos el formato del correo ni normalizamos códigos de país: «809-555-0101» y «8095550101» siguen siendo cadenas distintas.

Listado 2.1

```
from __future__ import annotations

def required(value: str) -> str:
    value = value.strip()
    if not value:
        raise ValueError("El campo no puede quedar vacío.")
    return value

class ContactNode:
    __slots__ = ("name", "email", "phone", "next")

    def __init__(self, name: str, email: str, phone: str):
        self.name = name
        self.email = email
        self.phone = phone
        self.next: ContactNode | None = None

    def __repr__(self):
        return f"{self.name} | {self.email} | {self.phone}"
```

`required` concentra una regla reutilizable y lanza `ValueError` si no se cumple. `ContactNode` agrupa los campos y comienza aislado, con `next` igual a `None`. `__repr__` ofrece una representación breve para la consola. Crear el nodo no lo incorpora a una lista: todavía falta conectarlo.

2.7.2 Contenedor y consultas

Continúa en `contacts.py` con la clase que administra los nodos. Los métodos públicos recibirán textos y crearán internamente nodos nuevos, lo que evita que una inserción reutilice por accidente un nodo ya enlazado.

Listado 2.2

```
class ContactLinkedList:
    def __init__(self):
        self.head: ContactNode | None = None
        self.tail: ContactNode | None = None
        self.length = 0

    def __len__(self):
        return self.length

    def is_empty(self):
        return self.head is None

    def __iter__(self):
        current = self.head
        while current is not None:
            yield current
            current = current.next

    def find(self, phone: str):
        phone = required(phone)
        for node in self:
            if node.phone == phone:
                return node
        return None

    def _new(self, name, email, phone):
        name, email, phone = map(required, (name, email, phone))
        if self.find(phone) is not None:
            raise ValueError("El teléfono ya existe.")
        return ContactNode(name, email, phone)
```

`__len__` permite escribir `len(agenda)` sin contar otra vez los nodos. `__iter__` permite usar `for node in agenda`. `find` devuelve el nodo encontrado o `None`; devolver un nodo existente no crea una copia.

`_new` es un auxiliar de uso interno. Primero limpia todos los campos y después comprueba la unicidad del teléfono. Si algo falla, la lista todavía no ha cambiado. El prefijo de guion bajo expresa una convención de uso; Python no lo convierte en un método privado inaccesible.

Precisión. Para consultar si la lista está vacía, usa `is_empty()` o `len(agenda) == 0`. No edites directamente los enlaces ni los teléfonos de los nodos devueltos por `find`: los métodos de la lista son los responsables de mantener las invariantes y la unicidad.

2.8 Agenda: insertar en los extremos y en medio

2.8.1 Agregar al inicio y al final

Continúa dentro de `ContactLinkedList`. `prepend` agrega al inicio; `append`, al final. Ambas operaciones deben resolver explícitamente el caso vacío.

Listado 2.3

```
def prepend(self, name, email, phone):
    node = self._new(name, email, phone)
    node.next = self.head
    self.head = node
    if self.tail is None:
        self.tail = node
    self.length += 1
    return node

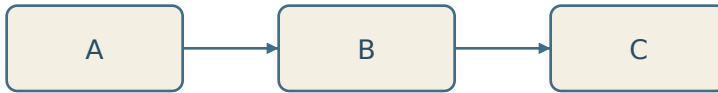
def append(self, name, email, phone):
    node = self._new(name, email, phone)
    if self.tail is None:
        self.head = node
    else:
        self.tail.next = node
    self.tail = node
    self.length += 1
    return node
```

Al insertar al inicio, guardamos el antiguo primer nodo en `node.next` antes de reemplazar `head`. Si reemplazáramos `head` primero y luego hiciéramos `node.next = self.head`, el nodo nuevo se apuntaría a sí mismo y perderíamos el acceso normal al resto de la secuencia.

Al agregar al final de una lista no vacía, la cola anterior apunta al nuevo nodo y luego `tail` se actualiza. Si estaba vacía, `head` también debe apuntar al nuevo nodo. El constructor ya dejó `node.next` en `None`.

2.8.2 Insertar después de un teléfono

ANTES



DESPUÉS



1. X.next = B.next
2. B.next = X

Figura 2.4 · Conservar el sucesor antes de cambiar el enlace.

Con una secuencia A, B, C, insertar X después de B debe producir A, B, X, C. Primero se conserva el sucesor de B en X.next; después B.next pasa a ser X. Cuando B era la cola, X se convierte en la nueva cola.

Listado 2.4

```

def insert_after(self, target_phone, name, email, phone):
    target = self.find(target_phone)
    if target is None:
        raise ValueError("No existe el teléfono de referencia.")
    node = self._new(name, email, phone)
    node.next = target.next
    target.next = node
    if self.tail is target:
        self.tail = node
    self.length += 1
    return node
  
```

La búsqueda del teléfono de referencia y la validación de unicidad pueden recorrer la lista. Solo las asignaciones de enlaces son constantes. Si el teléfono objetivo es el de head, insertamos en la segunda posición: no debemos llamar a prepend, porque eso insertaría antes del objetivo.

Actividad guiada. Dibuja una lista con un solo contacto e inserta otro después de él. Comprueba que head no cambia, tail pasa al nuevo nodo y length pasa de 1 a 2.

2.9 Agenda: eliminar sin perder la secuencia

Eliminar significa retirar un nodo de la estructura. Los demás deben seguir siendo alcanzables desde head. Nuestros métodos devuelven el nodo retirado para poder informar qué registro se eliminó.

2.9.1 Retirar el primer contacto

Listado 2.5

```
def delete_start(self):
    if self.head is None:
        raise ValueError("La lista está vacía.")
    removed = self.head
    self.head = removed.next
    if self.head is None:
        self.tail = None
    removed.next = None
    self.length -= 1
    return removed
```

Guardamos una referencia al nodo retirado, adelantamos head y corregimos tail si la lista quedó vacía. Luego desconectamos removed.next y reducimos el contador. Ninguna de esas acciones depende de n: el costo es $O(1)$.

2.9.2 Retirar el último contacto

Listado 2.6

```
def delete_end(self):
    if self.head is None:
        raise ValueError("La lista está vacía.")
    if self.head is self.tail:
        return self.delete_start()
    previous = self.head
    while previous.next is not self.tail:
        previous = previous.next
    removed = self.tail
    previous.next = None
    self.tail = previous
    self.length -= 1
    return removed
```

Si había un único nodo, usamos delete_start. En otro caso, previous avanza hasta que previous.next es tail. Ese es el penúltimo nodo: se convertirá en la nueva cola. La condición del ciclo mantiene el recorrido dentro de una lista que sabemos no vacía y con más de un elemento.

Disponer de tail evita buscar al último, pero no revela quién es su anterior. Por eso delete_end sigue siendo $O(n)$ en la lista simple. El último nodo ya tenía next igual a None, de modo que no necesita otra desconexión hacia adelante.

2.9.3 Retirar por teléfono

Listado 2.7

```
def delete(self, phone):
    phone = required(phone)
    previous = None
    current = self.head
    while current is not None and current.phone != phone:
        previous = current
        current = current.next
    if current is None:
        raise ValueError("No existe ese teléfono.")
    if previous is None:
        return self.delete_start()
    previous.next = current.next
    if current is self.tail:
        self.tail = previous
    current.next = None
    self.length -= 1
    return current
```

Durante la búsqueda, `previous` queda un paso detrás de `current`. Si `current` es `head`, delegamos en `delete_start`. En los demás casos, el anterior salta sobre el nodo retirado. Si eliminamos `tail`, el anterior pasa a ser `tail`. El contador se reduce una sola vez.

Precisión. Desconectar un nodo no implica que desaparezca inmediatamente de memoria. Si una variable conserva el objeto devuelto, este sigue disponible. Python administra su memoria; nuestra responsabilidad aquí es retirar correctamente las referencias de la lista.

2.10 Agenda: edición segura y primera ejecución

2.10.1 Validar antes de modificar

Continúa dentro de `ContactLinkedList` con el último método. Editar cambia los datos del nodo, pero no sus enlaces ni su posición.

Listado 2.8

```
def edit(self, phone, *, name=None, email=None, new_phone=None):
    node = self.find(phone)
    if node is None:
        raise ValueError("No existe ese teléfono.")
    next_name = node.name if name is None else required(name)
    next_email = node.email if email is None else required(email)
    next_phone = node.phone if new_phone is None else required(new_phone)
    other = self.find(next_phone)
    if other is not None and other is not node:
        raise ValueError("El teléfono ya existe.")
    node.name, node.email, node.phone = next_name, next_email, next_phone
    return node
```

El asterisco de la firma obliga a pasar los cambios por nombre, como `name="Ana"`. `None` significa conservar el valor existente. Una cadena vacía explícita es inválida. Permitir el mismo teléfono del nodo actual no viola la unicidad; por eso distinguimos `other` is node de otro contacto con ese teléfono.

Todos los valores nuevos se validan antes de asignarlos. Si el nuevo teléfono ya existe, también se conserva el nombre anterior. Esta decisión evita una edición parcial en la que el usuario recibe un error, pero algunos campos ya cambiaron.

2.10.2 Experimento independiente

Guarda temporalmente este código en `demo_contacts.py`, dentro de la misma carpeta. Ejecuta `python demo_contacts.py` desde esa carpeta.

Listado 2.9

```
from contacts import ContactLinkedList

agenda = ContactLinkedList()
agenda.append("Leo", "leo@example.com", "101")
agenda.append("José", "jose@example.com", "102")
agenda.prepend("María", "maria@example.com", "103")
agenda.insert_after("101", "Mateo", "mateo@example.com", "104")
print([node.name for node in agenda])
# ['María', 'Leo', 'Mateo', 'José']

agenda.delete_start()
agenda.delete_end()
agenda.edit("104", name="Mateo Pérez")
print([node.name for node in agenda])
# ['Leo', 'Mateo Pérez']
print(len(agenda)) # 2
```

Las comprensiones construyen listas de Python solo para mostrar el resultado. El almacenamiento de agenda continúa siendo una cadena de `ContactNode`. Intenta agregar otro contacto con teléfono 101 y comprueba que la operación produzca `ValueError` sin modificar los dos contactos existentes.

2.11 Tickets: modelo y reglas de atención

La segunda práctica incorpora enlaces en ambos sentidos. Un ticket representa una solicitud de ayuda. Su identificador distingue la solicitud; cliente, asunto, descripción y estado explican quién necesita atención y por qué.

Campo	Regla del proyecto
ticket_id	Cadena UUID generada al crear el nodo; no se edita.
client, subject, description	Textos requeridos, sin espacios vacíos en los extremos.
status	Uno de tres valores: open, in progress o close.
next y prev	Referencias al siguiente y al anterior, o None en los extremos.

Los tickets normales entran por tail y se atienden desde head. Esto produce orden FIFO, primero en entrar, primero en salir, mientras solo se usen esas operaciones. Los urgentes entran por head para adelantarse a los demás. Si ingresan varios urgentes sucesivos, el más reciente queda primero; no estamos implementando una cola de prioridad ordenada por niveles.

Retirar el último y cancelar por ID son operaciones adicionales de administración. Marcar un ticket como close modifica su estado, pero no lo elimina. Atenderlo en esta simulación lo retira y muestra sus datos; no conserva un historial permanente ni impide atender tickets cuyo estado ya sea close.

2.11.1 Comienzo de tickets.py

Listado 2.10

```
from __future__ import annotations
from uuid import uuid4
from contacts import required

VALID_STATUSES = {"open", "in progress", "close"}

def normalize_status(value):
    value = " ".join(required(value).lower().split())
    if value not in VALID_STATUSES:
        raise ValueError("Estado válido: open, in progress o close.")
    return value

class TicketNode:
    __slots__ = ("ticket_id", "client", "subject", "description",
                 "status", "prev", "next")

    def __init__(self, client, subject, description, status):
        self.ticket_id = str(uuid4())
        self.client = required(client)
        self.subject = required(subject)
        self.description = required(description)
        self.status = normalize_status(status)
        self.prev: TicketNode | None = None
        self.next: TicketNode | None = None

    def __repr__(self):
        return (f"{self.ticket_id} | {self.client} | {self.subject} | "
                f"{self.status} | {self.description}")
```

uuid4 genera identificadores aleatorios que representamos como texto. Cada ejecución mostrará valores diferentes. La probabilidad de coincidencia es muy baja; aun así, el contenedor comprobará que el identificador no exista en la lista actual.

normalize_status convierte « IN PROGRESS » en «in progress». Se conserva la palabra close usada en la clase grabada, aunque otras aplicaciones podrían elegir closed. El programa acepta solo el vocabulario definido por VALID_STATUSES.

En esta edición la lista recibe los datos y crea el nodo internamente. La clase grabada construye el nodo antes de insertarlo. Ambas organizan los mismos enlaces; la interfaz del manual evita insertar dos veces el mismo objeto y mantiene la validación en un solo lugar.

2.12 Tickets: recorridos y búsqueda

2.12.1 Cabecera, cola y dirección

Continúa en tickets.py con el contenedor. Un recorrido hacia adelante empieza en head y sigue next. El inverso empieza en tail y sigue prev.

Listado 2.11

```
class TicketDoublyLinkedList:
    def __init__(self):
        self.head: TicketNode | None = None
        self.tail: TicketNode | None = None
        self.length = 0

    def __len__(self):
        return self.length

    def is_empty(self):
        return self.head is None

    def __iter__(self):
        current = self.head
        while current is not None:
            yield current
            current = current.next

    def backward(self):
        current = self.tail
        while current is not None:
            yield current
            current = current.prev

    def find(self, ticket_id):
        ticket_id = required(ticket_id)
        for node in self:
            if node.ticket_id == ticket_id:
                return node
        return None
```

Listado 2.11 - continuación

```
def _new(self, client, subject, description, status):
    node = TicketNode(client, subject, description, status)
    if self.find(node.ticket_id) is not None:
        raise ValueError("ID repetido. Intente crear otro ticket.")
    return node
```

Ambos recorridos deben visitar la misma cantidad de nodos en orden contrario. Que el recorrido hacia adelante funcione no demuestra que prev esté bien: una inserción puede actualizar next y olvidar la conexión inversa.

find sigue siendo una búsqueda lineal por identificador. Tener enlaces dobles no crea un índice de UUID. `_new` valida los datos antes de incorporar el nodo y rechaza una coincidencia de ID; el usuario puede reintentar la creación. A diferencia del ciclo de regeneración de la clase grabada, esta variante realiza un solo intento por llamada.

Precisión. No cambies la estructura mientras consumes uno de sus generadores. Los ejemplos suponen que cada recorrido termina antes de insertar o retirar nodos. El menú cumple esta condición porque procesa una opción a la vez.

2.13 Tickets: insertar y conservar los enlaces dobles

2.13.1 Normal al final, urgente al inicio

Continúa dentro de `TicketDoublyLinkedList`.

Listado 2.12

```
def append(self, client, subject, description, status="open"):
    node = self._new(client, subject, description, status)
    node.prev = self.tail
    if self.tail is None:
        self.head = node
    else:
        self.tail.next = node
    self.tail = node
    self.length += 1
    return node

def prepend(self, client, subject, description, status="open"):
    node = self._new(client, subject, description, status)
    node.next = self.head
    if self.head is None:
        self.tail = node
    else:
        self.head.prev = node
    self.head = node
    self.length += 1
    return node
```

En `append`, `node.prev` conserva la cola anterior. Si existía, esa cola debe actualizar su `next`. En `prepend`, `node.next` conserva la cabecera anterior y esa cabecera debe actualizar su `prev`. El constructor deja el enlace exterior del nuevo extremo en `None`.

En el caso vacío, la misma instancia queda como `head` y `tail`. No hay un vecino al cual actualizar; ambos enlaces siguen vacíos. El contador aumenta una sola vez en cualquiera de los dos casos.

2.13.2 Inserción después de un ID

Listado 2.13

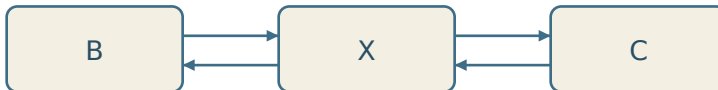
```
def insert_after(self, target_id, client, subject, description,
                status="open"):
    target = self.find(target_id)
    if target is None:
        raise ValueError("No existe el ticket de referencia.")
    node = self._new(client, subject, description, status)
    following = target.next
    node.prev = target
    node.next = following
    target.next = node
    if following is None:
        self.tail = node
    else:
        following.prev = node
    self.length += 1
    return node
```

Al insertar X entre B y C intervienen cuatro enlaces: X.prev apunta a B, X.next a C, B.next a X y C.prev a X. following conserva C antes de modificar B.next. Si no hay C, X pasa a ser tail.

ANTES



DESPUÉS



X.prev = B; X.next = C; B.next = X; C.prev = X

Figura 2.5 · Una inserción doble actualiza cuatro enlaces.

Actividad guiada. Repite el razonamiento cuando B es head y cuando B es tail. En ambos casos X queda después de B. Comprueba que insertar después del primer nodo no modifica head.

2.14 Tickets: retirar y editar

2.14.1 Una operación interna de desconexión

Los tres tipos de eliminación comparten los mismos cambios. Para no repetirlos, `_unlink` recibe un nodo que pertenece a esta lista. Solo los métodos internos deben llamarlo y cumplir esa condición.

Listado 2.14

```
def _unlink(self, node):
    previous, following = node.prev, node.next
    if previous is None:
        self.head = following
    else:
        previous.next = following
    if following is None:
        self.tail = previous
    else:
        following.prev = previous
    node.prev = node.next = None
    self.length -= 1
    return node

def delete_start(self):
    if self.head is None:
        raise ValueError("La lista está vacía.")
    return self._unlink(self.head)

def delete_end(self):
    if self.tail is None:
        raise ValueError("La lista está vacía.")
    return self._unlink(self.tail)

def delete(self, ticket_id):
    node = self.find(ticket_id)
    if node is None:
        raise ValueError("No existe ese ticket.")
    return self._unlink(node)
```

Si no hay anterior, retiramos la cabecera y adelantamos head. Si sí hay anterior, su next debe saltar al siguiente. La segunda mitad hace lo simétrico: sin siguiente cambiamos tail; con siguiente corregimos su prev. Finalmente, ambos enlaces del nodo retirado quedan en None.

Este esquema cubre nodo inicial, final, interior y único. Si era el único nodo, previous y following valen None, por lo que head y tail se vuelven None. No es necesario escribir otra rama especial para ese caso.

`delete_start` y `delete_end` reciben directamente un extremo, por lo que cuestan $O(1)$. `delete(ticket_id)` primero busca y luego desconecta; el método completo cuesta $O(n)$. La facilidad de desconectar no elimina el costo de localizar.

2.14.2 Edición sin cambiar el identificador

Listado 2.15

```
def edit(self, ticket_id, *, client=None, subject=None,
        description=None, status=None):
    node = self.find(ticket_id)
    if node is None:
        raise ValueError("No existe ese ticket.")
    new_client = node.client if client is None else required(client)
    new_subject = node.subject if subject is None else required(subject)
    new_desc = node.description
    if description is not None:
        new_desc = required(description)
    new_status = node.status if status is None else normalize_status(status)
    node.client, node.subject = new_client, new_subject
    node.description, node.status = new_desc, new_status
    return node
```

Como en la agenda, primero calculamos y validamos todos los cambios. Solo entonces actualizamos el nodo. No creamos un ticket de reemplazo: conservamos su identidad, su ID y sus vecinos. Un estado inválido deja intactos los demás campos.

2.14.3 Experimento independiente

Guarda este bloque en `demo_tickets.py` y ejecútalo con `python demo_tickets.py`.

Listado 2.16

```
from tickets import TicketDoublyLinkedList

queue = TicketDoublyLinkedList()
leo = queue.append("Leo", "Acceso", "No puedo iniciar sesión")
queue.append("José", "Impresora", "No imprime")
queue.prepend("María", "Urgente", "Servicio detenido")
queue.insert_after(leo.ticket_id, "Mateo", "Red", "Sin conexión")
print([node.client for node in queue])
# ['María', 'Leo', 'Mateo', 'José']
print([node.client for node in queue.backward()])
# ['José', 'Mateo', 'Leo', 'María']
print(queue.delete_start().client) # María
print(queue.delete_end().client)  # José
queue.edit(leo.ticket_id, status="IN PROGRESS")
print(leo.status)                  # in progress
```

2.15 Menús de consola: compartir la interacción

2.15.1 Entrada y presentación

Ya funcionan las estructuras. Ahora `main.py` permitirá operarlas sin escribir llamadas manualmente. El menú solicita datos y muestra resultados; las clases siguen decidiendo qué operaciones son válidas.

Listado 2.17

```

from contacts import ContactLinkedList, required
from tickets import TicketDoublyLinkedList

def ask(label):
    while True:
        try:
            return required(input(label + ": "))
        except ValueError as error:
            print(error)

def optional(label):
    return input(label + " [Enter conserva]: ").strip() or None

def show(nodes):
    found = False
    for node in nodes:
        found = True
        print(node)
    if not found:
        print("Sin registros.")

def contact_data():
    return ask("Nombre"), ask("Correo"), ask("Teléfono")

def ticket_data():
    return (ask("Cliente"), ask("Asunto"), ask("Descripción"),
            ask("Estado: open / in progress / close"))

```

ask insiste hasta recibir texto no vacío. optional traduce Enter a None para conservar un dato durante la edición. show recibe cualquier iterable de nodos, así que sirve tanto para la agenda como para los dos sentidos de la lista doble.

El estado se valida en tickets.py. Si no es válido, el menú informa el error y vuelve a ofrecer sus opciones; el ticket no se incorpora. Este comportamiento evita duplicar la regla de estados en la interfaz y en el modelo.

2.15.2 Menú de contactos

Continúa en main.py. Este bloque es una función de módulo: no lo pongas dentro de ContactLinkedList.

Listado 2.18

```

def contacts_menu(agenda):
    while True:
        print("\nCONTACTOS: 1 inicio, 2 final, 3 después de teléfono")
        print("4 borrar primero, 5 borrar último, 6 borrar por teléfono")
        print("7 buscar, 8 editar, 9 listar, 0 volver")

```

Listado 2.18 - continuación

```

option = input("> ").strip()
try:
    if option == "0":
        return
    elif option in {"1", "2"}:
        method = agenda.prepend if option == "1" else agenda.append
        print(method(*contact_data()))
    elif option == "3":
        target = ask("Teléfono de referencia")
        print(agenda.insert_after(target, *contact_data()))
    elif option == "4":
        print("Retirado:", agenda.delete_start())
    elif option == "5":
        print("Retirado:", agenda.delete_end())
    elif option == "6":
        print("Retirado:", agenda.delete(ask("Teléfono")))
    elif option == "7":
        print(agenda.find(ask("Teléfono")) or "No encontrado.")
    elif option == "8":
        target = ask("Teléfono actual")
        print(agenda.edit(target, name=optional("Nombre"),
                        email=optional("Correo"),
                        new_phone=optional("Nuevo teléfono")))
    elif option == "9":
        show(agenda)
    else:
        print("Opción no válida.")

```

Listado 2.18 - continuación

```

except ValueError as error:
    print("No se realizó la operación:", error)

```

method guarda una referencia al método que se va a invocar. El asterisco de *contact_data() desempaqueta los tres valores para entregarlos como argumentos. No crea ni recorre nodos de la lista.

La captura de ValueError mantiene abierto el programa cuando el usuario busca un teléfono inexistente, intenta eliminar una lista vacía o introduce un duplicado. Imprimir el mensaje concreto del error facilita entender lo sucedido.

2.15.3 Menú de tickets

Listado 2.19

```
def tickets_menu(queue):
    while True:
        print("\nTICKETS: 1 normal, 2 urgente, 3 después de ID")
        print("4 atender primero, 5 retirar último, 6 cancelar por ID")
        print("7 buscar, 8 editar, 9 adelante, 10 atrás, 0 volver")
        option = input("> ").strip()
        try:
            if option == "0":
                return
            elif option in {"1", "2"}:
                method = queue.append if option == "1" else queue.prepend
                print(method(*ticket_data()))
            elif option == "3":
                target = ask("ID de referencia")
                print(queue.insert_after(target, *ticket_data()))
            elif option == "4":
                print("Atendido y retirado:", queue.delete_start())
            elif option == "5":
                print("Retirado:", queue.delete_end())
            elif option == "6":
                print("Cancelado:", queue.delete(ask("ID")))
            elif option == "7":
                print(queue.find(ask("ID")) or "No encontrado.")
            elif option == "8":
                target = ask("ID")
                print(queue.edit(target, client=optional("Cliente"),
                                subject=optional("Asunto"),
                                description=optional("Descripción"),
                                status=optional("Estado")))
            elif option == "9":
                show(queue)
            elif option == "10":
```

Listado 2.19 - continuación

```
                show(queue.backward())
            else:
                print("Opción no válida.")
        except ValueError as error:
            print("No se realizó la operación:", error)
```

Para las opciones por identificador, copia el UUID completo mostrado al listar. La opción 9 avanza desde head; la 10 retrocede desde tail. La opción 4 atiende y retira el primero; la 5 retira el último sin afirmar que sea el más antiguo ni el más urgente.

2.15.4 Punto de entrada

Añade este último bloque a main.py. Las dos listas se crean una sola vez, antes del ciclo principal, de modo que conservan sus datos al regresar de un submenú.

Listado 2.20

```
def main():
    agenda = ContactLinkedList()
    queue = TicketDoublyLinkedList()
    while True:
        option = input("\n1 Contactos | 2 Tickets | 0 Salir: ").strip()
        if option == "0":
            break
        elif option == "1":
            contacts_menu(agenda)
        elif option == "2":
            tickets_menu(queue)
        else:
            print("Opción no válida.")

if __name__ == "__main__":
    main()
```

Abre una terminal dentro de modulo2_listas y ejecuta el siguiente comando. En algunos equipos se utiliza python3 en lugar de python.

Listado 2.21

```
python main.py
```

Si aparece `ModuleNotFoundError`, comprueba que los tres archivos estén en la misma carpeta y que sus nombres coincidan. Si aparece `IndentationError`, revisa la sangría: los métodos pertenecen a su clase, mientras que las funciones de `main.py` están al nivel del módulo.

2.16 Sesión práctica de principio a fin

2.16.1 Recorrido de la agenda

1. Entra al menú de contactos. Lista los registros antes de crear ninguno: debe aparecer «Sin registros».
2. Agrega al final a Leo, teléfono 101, y a José, teléfono 102. Usa correos de ejemplo no vacíos.
3. Agrega al inicio a María, teléfono 103. Inserta a Mateo, teléfono 104, después del 101. El orden debe ser María, Leo, Mateo, José.
4. Intenta agregar otro contacto con teléfono 101. Debe rechazarse y conservarse el orden anterior.
5. Retira el primero y el último. Quedan Leo y Mateo. Edita a Mateo y conserva su correo pulsando Enter.
6. Intenta editar a Mateo con teléfono 101. Debe rechazarse sin cambiar sus datos. Elimina los dos contactos y vuelve a intentar retirar uno.

2.16.2 Recorrido de los tickets

1. Crea dos tickets normales para Leo y José. Guarda el ID de Leo.
2. Crea un ticket urgente para María. Lista hacia adelante y confirma María, Leo, José.
3. Inserta a Mateo después del ID de Leo. Comprueba los órdenes María, Leo, Mateo, José y José, Mateo, Leo, María.
4. Atiende al primero y retira al último. Deben salir María y José, respectivamente.
5. Edita el estado de Leo a in progress. Intenta después un estado distinto de los tres válidos y confirma que el anterior se conserva.
6. Cancela a Leo por su ID y retira el único ticket restante. Comprueba que ambos recorridos queden vacíos.

2.16.3 Qué observar, además de la pantalla

Después de cada modificación, el tamaño debe coincidir con la cantidad de registros visitados. En una lista de un elemento, head y tail deben ser el mismo nodo. En la doble, el primer prev y el último next deben ser None. Estas comprobaciones explican el comportamiento; no dependen del texto exacto de la interfaz.

Una aplicación en memoria como esta es apropiada para aprender estructuras. Agregar persistencia, historial de atención, formatos telefónicos o búsqueda indexada sería una ampliación con nuevas reglas; no debe confundirse con el objetivo de enlazar correctamente los nodos.

2.17 Comparar el costo real de ambos proyectos

Método o acción	Agenda simple	Tickets dobles
prepend / append completos	$O(n)$: teléfono único.	$O(n)$: comprobación del ID generado.
insert_after completo	$O(n)$: búsqueda y validación.	$O(n)$: búsqueda y validación.
delete_start	$O(1)$.	$O(1)$.
delete_end	$O(n)$.	$O(1)$.
find / delete por clave / edit	$O(n)$.	$O(n)$.
Recorrido completo	$O(n)$, solo hacia adelante.	$O(n)$ en cualquiera de los dos sentidos.
len / is_empty	$O(1)$.	$O(1)$.

Las validaciones no son un detalle ajeno al análisis: forman parte del trabajo solicitado al método. Construir n contactos mediante append con búsqueda de duplicados puede realizar $0 + 1 + 2 + \dots + (n - 1)$ comparaciones. El total es $n(n - 1)/2$, es decir, $O(n^2)$, aunque cada enlace nuevo solo requiera unas asignaciones.

La memoria de los nodos es $O(n)$ en ambos proyectos si los campos tienen tamaño acotado. La lista doble almacena una referencia adicional por nodo. Una búsqueda y una eliminación utilizan $O(1)$ memoria auxiliar; convertir todos los resultados a list, como hacemos para inspeccionar pruebas, requiere $O(n)$ espacio adicional.

2.17.1 Cuando usar cada representación

Una lista simple sirve para una pila que agrega y retira por el inicio, o una cola que agrega por tail y retira por head. Una doble resulta útil cuando necesitamos retroceder o retirar un nodo del que ya tenemos referencia. En ambos casos, buscar repetidamente por un dato sin índice puede dominar el tiempo total.

El historial de navegación y deshacer/rehacer ilustran el recorrido en ambos sentidos, aunque una aplicación real puede implementarlos con otras estructuras. Una caché LRU puede combinar lista doble e índice hash: el índice localiza el nodo y la lista modifica su orden. La rapidez de esa combinación depende de ambos componentes, no solo de los enlaces.

Si se añadiera un diccionario auxiliar por teléfono o ID, las búsquedas podrían ser $O(1)$ esperado. También habría que mantener el diccionario al insertar, editar y eliminar. Este manual conserva la búsqueda lineal para estudiar explícitamente los recorridos.

2.18 Extensión conceptual: listas circulares

Una lista circular representa secuencias que vuelven al inicio, como turnos repetidos o una reproducción cíclica. La circularidad es intencional y necesita invariantes propias; un ciclo accidental en una lista no circular es un error.

2.18.1 Reglas de cierre

En una circular simple no vacía, `tail.next` is `head`. En una circular doble se añade `head.prev` is `tail`. Si solo hay un nodo, sus enlaces circulares apuntan al propio nodo. En la lista vacía seguimos usando `head` y `tail` iguales a `None`.

Asignar `tail.next = head` guarda una referencia al objeto que `head` señala en ese momento. Si `head` cambia después, `tail.next` no se actualiza automáticamente. Cada operación que cambie un extremo debe reconstruir los enlaces de cierre correspondientes.

2.18.2 Un recorrido que termina

El siguiente ejemplo es conceptual y se usa exclusivamente con una lista circular simple válida. No lo agregues a los proyectos anteriores, cuyas colas terminan en None.

Listado 2.22

```
def circular_nodes(head):  
    if head is None:  
        return  
    current = head  
    while True:  
        yield current  
        current = current.next  
        if current is head:  
            break
```

El ciclo visita el nodo inicial una vez y se detiene cuando está a punto de repetirlo. Una alternativa es realizar exactamente `length` pasos, siempre que el contador sea correcto. La condición `while current is not None` no detendría un recorrido circular no vacío.

2.18.3 Inserción circular al inicio, paso a paso

En una lista vacía, crea el nodo, asigna `head` y `tail` a ese nodo y conecta su `next` consigo mismo. En una lista no vacía, conecta `node.next` con el antiguo `head`, asigna `head = node` y actualiza `tail.next = head`. En ambos casos incrementa `length`.

Para eliminar el único nodo, restaura la lista vacía. Para retirar el primero cuando hay varios, adelanta `head` y vuelve a conectar `tail.next` con el nuevo `head`. En una circular doble debes corregir también `head.prev`. Estos pasos no convierten automáticamente los métodos del proyecto en métodos circulares: sus búsquedas y eliminaciones también dependen de la terminación en None.

2.19 Pruebas: demostrar que los enlaces se conservan

2.19.1 Matriz mínima de casos

Estado inicial	Operación	Qué verificar
Vacía	Retirar primero o último.	Error y extremos intactos.
Vacía	Insertar.	head is tail, longitud 1.
Un nodo	Retirar ese nodo.	Ambos extremos None, longitud 0.
Varios nodos	Insertar tras head o tail.	Posición correcta y cola actualizada.
Varios nodos	Retirar interior.	Sus vecinos quedan conectados.
Varios nodos	Buscar o eliminar clave ausente.	Resultado previsto y lista intacta.
Varios nodos	Edición inválida.	Ningún campo cambia parcialmente.
Lista doble	Recorrer en ambos sentidos.	Secuencias inversas exactas.

2.19.2 Comprobaciones que puedes ejecutar

Crea `test_lists.py` en la misma carpeta. Este bloque comprueba estructura, comportamiento y conservación de datos después de errores. El conjunto `seen` permite detectar un ciclo accidental sin quedar atrapado en un recorrido infinito.

Listado 2.23

```

from contacts import ContactLinkedList
from tickets import TicketDoublyLinkedList

def check(sequence, double=False):
    seen = set()
    nodes = []
    current, previous = sequence.head, None
    while current is not None:
        assert id(current) not in seen, "Ciclo accidental"
        seen.add(id(current))
        if double:
            assert current.prev is previous
            nodes.append(current)
            previous, current = current, current.next
    assert len(nodes) == len(sequence)
    assert sequence.tail is (nodes[-1] if nodes else None)
    assert (sequence.head is None) == (len(sequence) == 0)
    if double:

```

Listado 2.23 - continuación

```

        assert list(sequence.backward()) == nodes[::-1]
    return nodes

def expect_error(operation):
    try:
        operation()
    except ValueError:
        return
    raise AssertionError("Se esperaba ValueError")

```

Listado 2.23 - continuación

```

def test_contacts():
    agenda = ContactLinkedList()
    check(agenda)
    expect_error(agenda.delete_start)
    expect_error(agenda.delete_end)
    first = agenda.append("Ana", "ana@example.com", "1")
    assert agenda.head is agenda.tail is first
    check(agenda)
    agenda.insert_after("1", "Beto", "beto@example.com", "2")
    agenda.prepend("Celia", "celia@example.com", "3")
    assert [n.phone for n in check(agenda)] == ["3", "1", "2"]
    expect_error(lambda: agenda.edit("1", name="Otro", new_phone="2"))
    assert first.name == "Ana" and first.phone == "1"
    removed = agenda.delete("1")
    assert removed is first and removed.next is None
    assert [n.phone for n in check(agenda)] == ["3", "2"]
    agenda.delete_end()
    check(agenda)
    agenda.delete_start()
    assert check(agenda) == []

def test_tickets():
    queue = TicketDoublyLinkedList()
    check(queue, True)
    expect_error(queue.delete_start)
    expect_error(queue.delete_end)
    a = queue.append("Ana", "Red", "Sin conexión")
    b = queue.append("Beto", "Correo", "No recibe")
    c = queue.prepend("Celia", "Urgente", "Servicio detenido")
    d = queue.insert_after(a.ticket_id, "Diego", "Cuenta", "Bloqueada")
    assert check(queue, True) == [c, a, d, b]

```

Listado 2.23 - continuación

```

    expect_error(lambda: queue.edit(a.ticket_id, client="Otro", status="x"))
    assert a.client == "Ana" and a.status == "open"
    assert queue.delete(d.ticket_id) is d
    assert d.prev is None and d.next is None
    assert check(queue, True) == [c, a, b]

```

Listado 2.23 - continuación

```

    assert queue.delete_start() is c
    check(queue, True)
    assert queue.delete_end() is b
    assert check(queue, True) == [a]
    assert queue.delete_end() is a
    assert check(queue, True) == []

if __name__ == "__main__":
    test_contacts()
    test_tickets()
    print("Todas las comprobaciones pasaron.")

```

Ejecuta `python test_lists.py` sin la opción `-O`, que desactiva `assert`. El mensaje final solo aparece si todas las comprobaciones terminan correctamente. Si falla una aserción, lee su línea, reproduce la operación con dos o tres nodos y dibuja los enlaces antes de cambiar el método.

Estas pruebas son un punto de partida. Añade casos de teléfonos repetidos, IDs inexistentes y varias inserciones consecutivas en los extremos. Para una prueba más amplia, compara una secuencia de operaciones con una lista de Python usada como modelo y revisa las invariantes después de cada paso.

2.20 Errores frecuentes y estrategia de depuración

Síntoma	Causa probable	Comprobación útil
El recorrido no termina.	Se creó un ciclo accidental.	Registrar identidad de nodos visitados.
Desaparecen registros al insertar.	Se sobrescribió el enlace al sucesor.	Guardar primero el sucesor en el nuevo nodo.
Hacia adelante funciona, hacia atrás falla.	No se actualizó un prev.	Verificar que <code>node.next.prev</code> sea <code>node</code> .
La lista parece vacía pero tail existe.	Se retiró el último nodo sin limpiar ambos extremos.	Caso de una sola instancia.
len no coincide con los registros.	Incremento o decremento omitido o duplicado.	Contar nodos visitados tras cada operación.
Un error deja datos modificados.	La edición asignó antes de validar todos los campos.	Preparar valores nuevos y asignar al final.

No depures con una colección grande desde el comienzo. Una lista de tres nodos permite distinguir primero, interior y último. Marca las variables locales con su función: `current` es el nodo visitado, `previous` su anterior y `following` el sucesor que necesitas conservar.

Al detectar un fallo, identifica primero la invariante violada. Después revisa la operación que debió mantenerla. Cambiar varias asignaciones a la vez dificulta descubrir cuál era la causa. En una doble, comprueba ambas direcciones después de corregir cualquier inserción o eliminación.

Precisión. Una implementación puede mostrar el resultado esperado y conservar un enlace incorrecto que falle más tarde. Las comprobaciones estructurales y las pruebas de los casos límite complementan la observación de la consola.

2.21 Actividades y respuestas orientativas

2.21.1 Actividades para resolver

Actividad 1. Explica por qué una lista enlazada es lineal aunque sus nodos no estén contiguos en memoria. Distingue orden lógico de ubicación física.

Actividad 2. Una lista simple conserva `head` y `tail`. Justifica por qué enlazar al final cuesta $O(1)$, pero retirar el último cuesta $O(n)$.

Actividad 3. Parte de A, B, C. Inserta X después de A, retira C y luego retira el primero. Escribe el orden final, `head`, `tail` y `length`.

Actividad 4. Un estudiante escribe `target.next = node` y después `node.next = target.next`. ¿Qué enlace produce? Corrige el orden de las dos instrucciones.

Actividad 5. En una lista doble, inserta X entre B y C. Enumera los cuatro enlaces que deben cambiar y explica qué ocurre si se omite C.`prev`.

Actividad 6. Recorre la lógica de `_unlink` cuando la lista solo contiene un ticket. ¿Qué valores tendrán `head`, `tail`, `node.prev`, `node.next` y `length` al terminar?

Actividad 7. Intenta cambiar el nombre y el teléfono de un contacto a un teléfono que ya existe. Diseña una prueba que demuestre que el nombre tampoco cambió.

Actividad 8. Llegan tickets normales A y B; después urgentes U1 y U2. Si todos entran mediante los métodos del manual, ¿en qué orden se atenderán desde `head`? ¿Se respeta FIFO entre los urgentes?

Actividad 9. Explica por qué el `append` completo de la agenda es $O(n)$. ¿Cuánto cuesta construir n contactos distintos mediante ese método sin índice auxiliar?

Actividad 10. Una lista circular tiene un único nodo. Dibuja sus enlaces en la variante simple y en la doble. Explica por qué un ciclo que espera None no termina.

Actividad 11. Implementa `count_status(status)` para contar tickets con un estado dado sin construir una lista adicional. Indica tiempo y memoria auxiliar.

Actividad 12. Como práctica integradora, agrega `move_to_front(ticket_id)` a la lista doble: debe mover el mismo nodo al inicio, conservar su ID y no alterar el número final de tickets. Prueba un nodo interior, tail, head, un único nodo y un ID ausente.

2.21.2 Respuestas comentadas

Respuesta 1. Es lineal porque cada posición tiene un sucesor lógico dentro de una secuencia, salvo el extremo correspondiente. La memoria puede estar dispersa; `next` representa el orden.

Respuesta 2. `tail` permite acceder al último nodo directamente y conectarlo con el nuevo. Para eliminarlo necesitamos su anterior; una lista simple no conserva `prev` y debe encontrarlo desde `head`.

Respuesta 3. Los estados son A, X, B, C; después A, X, B; finalmente X, B. `head` apunta a X, `tail` a B y `length` vale 2.

Respuesta 4. Tras la primera asignación, `target.next` ya es `node`. La segunda hace que `node.next` sea `node`: aparece un ciclo y se pierde el enlace al antiguo sucesor. Primero escribe `node.next = target.next` y después `target.next = node`.

Respuesta 5. `X.prev = B`, `X.next = C`, `B.next = X` y `C.prev = X`. Si se omite el último cambio, el recorrido inverso de C salta directamente a B y no visita X.

Respuesta 6. `previous` y `following` son None, de modo que los dos extremos se asignan a None. Ambos enlaces del retirado quedan None y `length` pasa a cero. El objeto retornado todavía puede usarse fuera de la lista.

Respuesta 7. Guarda los valores anteriores, llama a `edit` con `name="Nuevo"` y `new_phone` igual al de otro nodo, captura `ValueError` y compara ambos campos con los originales. Validar todos los candidatos antes de asignar evita la actualización parcial.

Respuesta 8. U2, U1, A, B. Los urgentes nuevos se colocan por delante de los urgentes anteriores, así que no mantienen FIFO entre sí. Para hacerlo, se podría diseñar una cola separada de urgentes con su propia regla de entrada y salida.

Respuesta 9. `_new` recorre la lista para comprobar la unicidad. La suma de los tamaños antes de las inserciones es $n(n - 1)/2$; el costo total es $O(n^2)$ bajo el modelo de campos acotados.

Respuesta 10. En la simple, `node.next` es el propio nodo; en la doble, también `node.prev`. No hay `None` dentro del ciclo, por lo que se necesita una condición de retorno al inicio o un conteo de pasos.

Respuesta 11. Normaliza primero el estado y suma coincidencias durante un único recorrido. Añade este método dentro de `TicketDoublyLinkedList`; es $O(n)$ en tiempo y $O(1)$ en memoria auxiliar.

Listado 2.24

```
def count_status(self, status):
    status = normalize_status(status)
    count = 0
    for node in self:
        if node.status == status:
            count += 1
    return count
```

Respuesta 12. Busca el nodo, resuelve el caso en que ya es head y desconéctalo con `_unlink`. Vuelve a enlazar esa misma instancia al inicio. No uses `prepend`, porque crearía otro nodo y otro ID. El siguiente método es una ampliación opcional dentro de la clase.

Listado 2.25

```
def move_to_front(self, ticket_id):
    node = self.find(ticket_id)
    if node is None:
        raise ValueError("No existe ese ticket.")
    if node is self.head:
        return node
    self._unlink(node)
    node.next = self.head
    self.head.prev = node
    self.head = node
    self.length += 1
    return node
```

Después de descartar que el nodo sea head, la lista tiene al menos dos elementos. Por eso, tras desconectarlo, todavía existe una cabecera cuyo `prev` puede actualizarse. `_unlink` reduce `length` y la reinserción lo incrementa: el total final no cambia. La búsqueda hace que el método sea $O(n)$.

2.22 Glosario y fuentes del capítulo

Término	Significado en este manual
Nodo / node	Objeto que reúne datos y enlaces.
Referencia	Vínculo que permite acceder a un objeto.
head / tail	Referencias al primer y al último nodo.
next / prev	Enlaces al siguiente y al anterior.
Invariante	Condición que debe conservar cada operación válida.
Recorrido / traversal	Visita sucesiva de nodos siguiendo enlaces.
FIFO	Primero en entrar, primero en salir.
UUID	Identificador que aquí se genera con uuid4 y se guarda como texto.
Complejidad auxiliar	Memoria adicional utilizada por una operación.
Desconexión / unlink	Retiro estructural de un nodo mediante actualización de enlaces.

2.22.1 Materiales de base

[S1] Ing. Leonardo Tavarez. «Listas Enlazadas: Fundamentos, Tipos y Complejidad». Presentación proporcionada para el módulo, 12 diapositivas. Fundamenta la comparación con arreglos, los tipos de listas, las operaciones, las aplicaciones y los errores habituales.

[S2] Transcripción suministrada en «Pasted text(20260917-000605).txt». Clase práctica sobre una agenda de contactos con lista simplemente enlazada en Python: modelo, validación, operaciones y menú.

[S3] Transcripción suministrada en «Pasted text (2)(5).txt». Clase práctica sobre un sistema de tickets con lista doblemente enlazada en Python: enlaces, operaciones, UUID, estados y recorridos en ambas direcciones.

2.22.2 Criterios de elaboración

El capítulo reorganiza las fuentes como una lección autónoma. Los diagramas son reconstrucciones didácticas. Se distingue el costo de enlazar del de buscar o validar. El código reúne los menús, crea los nodos dentro de las listas, centraliza la desconexión doble en `_unlink` y valida las ediciones antes de modificar campos. La circularidad se estudia por separado de los dos proyectos no circulares.